

EdgePulse: A Runtime Observability Framework for Quantized Large Language Models on Consumer Edge Devices

Author: Muhammad Assad Ullah

Affiliation: Independent Researcher

Contact: asadullahaj602@gmail.com

Code: <https://github.com/assassinaj602/edgepulse>

Dataset: <https://github.com/assassinaj602/edgepulse/tree/main/research/experiment>

Date: October 2026

Abstract

On-device AI execution lacks unified runtime observability across mobile platforms. Existing tooling typically focuses on isolated model profiling or single-engine metrics, failing to provide a synchronized view of memory consumption, thermal dynamics, battery draw, and inference latency during execution on consumer edge hardware.

We present **EdgePulse**, an open-source framework for on-device AI runtime observability. EdgePulse captures structured **InferenceTrace** telemetry on Android (Kotlin) and iOS (Swift) via native platform channels. Collected metrics include resident memory (PSS via `Debug.getPss()`), OS thermal state (`PowerManager.currentThermalStatus`), battery current draw (`BatteryManager.CURRENT_NOW`), CPU utilization (`/proc/stat`), and per-inference latency.

We evaluate EdgePulse through a real-device empirical study on a physical Tecno CH7n handset (MediaTek Helio G35, 4 GB RAM, Android 12, API 31) across three runtimes: TensorFlow Lite (MobileNet-V3-Small), ONNX Runtime (ResNet18), and GGUF/llama.cpp (TinyLlama 1.1B Q4). Analyzing 90 baseline traces and 60 sustained-load traces (15 minutes per workload) reveals three primary empirical findings:

1. Baseline mean latency varied by $12\times$ across runtimes (148 ms for ONNX to 1779 ms for GGUF), with peak RSS spanning from 276 MB (TFLite) to 905 MB (GGUF).
2. Sustained MobileNet inference over 15 minutes (4,590 iterations) exhibited no latency degradation (rolling mean 143–166 ms).
3. Sustained TinyLlama LLM decoding over 15 minutes produced a **+16.5% latency degradation** (1829 ms $\rightarrow$ 2131 ms) while `PowerManager.currentThermalStatus` reported nominal throughout the entire experiment.

Finding (3) highlights a thermal-API blind spot: standard OS-level thermal APIs can fail to indicate underlying hardware thermal throttling on budget ARM SoCs. These results demonstrate the necessity of direct, application-level runtime telemetry for edge AI deployments.

1. Introduction

Deploying quantized Large Language Models (LLMs) and optimized deep neural networks directly to consumer edge devices reduces cloud infrastructure overhead, protects user privacy, and enables offline capability. Frameworks such as TensorFlow Lite, ONNX Runtime, and llama.cpp make complex neural network inference viable on commodity mobile hardware.

However, executing neural networks on consumer mobile hardware introduces operational challenges that do not exist in server environments. Mobile devices rely on passive thermal dissipation and operate under variable system resource availability. Under continuous compute loads, mobile System-on-Chips (SoCs) undergo dynamic voltage and frequency scaling (DVFS) or CPU core shut-down to maintain thermal safety boundaries. Concurrently, operating system memory management daemons evict background processes or throttle applications under high memory pressure.

1.1 The Observability Deficit

Model evaluation standardly relies on isolated micro-benchmarks or static accuracy validation. These measurements do not capture runtime behavior under sustained operational conditions on physical devices. Existing mobile profilers (such as Android Studio Profiler or Xcode Instruments) offer low-level OS counters but lack native integration with model execution loops and cross-platform unified logging APIs.

This creates an observability deficit: developers cannot correlate hardware state transitions (such as subtle thermal throttling or memory growth) directly with inference latency, battery drain, and process memory footprints in production cross-platform applications.

1.2 Contributions

This paper makes the following contributions:

1. **EdgePulse Framework:** An open-source runtime observability engine (`edgepulse_core`) and Flutter native plugin (`edgepulse`) providing unified telemetry collection across Android (Kotlin) and iOS (Swift) via low-overhead platform channels.
2. **Real-Device Empirical Study:** An empirical evaluation characterizing three runtime engines (TFLite, ONNX Runtime, and llama.cpp/GGUF) on physical Android hardware (Tecno CH7n, MediaTek Helio G35, Android 12).
3. **Identification of the Thermal-API Blind Spot:** Empirical evidence demonstrating a +16.5% latency degradation during sustained 15-minute LLM decoding while OS-level thermal APIs continuously report `nominal` state.
4. **Open Artifacts & Test Suites:** Full dataset, reproducible integration test suites, and analysis scripts released under the MIT license.

2. Background and Related Work

2.1 On-Device AI Runtimes

Mobile neural network inference relies on specialized runtime execution engines:

- **TensorFlow Lite (TFLite)**: Targeted execution engine supporting post-training quantization and platform hardware delegates (NNAPI, GPU).
- **ONNX Runtime**: Cross-platform engine supporting heterogeneous execution providers and automatic graph optimization.
- **llama.cpp / GGUF**: High-performance C/C++ matrix execution framework optimized for quantized LLMs (such as 4-bit and 8-bit GGUF formats) on CPU architectures.

2.2 Thermal Dissipation and Memory Pressure

Consumer smartphones depend on passive thermal conduction through the device enclosure. Sustained matrix multiplication workloads raise SoC junction temperatures. When temperatures exceed hardware safety limits, kernel-level thermal governors lower clock frequencies or gate active core frequencies.

Additionally, memory pressure from quantized LLM execution presents operational risks. Large working sets during prompt processing and Key-Value (KV) cache generation elevate process Proportional Set Size (PSS), increasing susceptibility to termination by system low-memory killers (LMK).

2.3 Existing Telemetry Approaches

Existing mobile profiling tools present distinct operational tradeoffs:

- **Framework Micro-benchmarks**: Tools such as the TFLite Benchmark CLI measure isolated iteration timing but do not capture OS thermal state transitions, battery current draw, or memory PSS deltas during application execution.
- **IDE Profilers**: Android Studio Profiler and Xcode Instruments offer comprehensive hardware metrics but require attached USB debugging sessions and GUI interaction, preventing continuous automated integration testing.

EdgePulse addresses these limitations by embedding lightweight, structured telemetry collection directly into the application runtime.

3. EdgePulse System Architecture

EdgePulse uses a modular architecture separating metric collection contracts, platform channel bindings, orchestration logic, and export formatters.

Flutter Application

EdgePulse Facade (EdgePulse class)

PlatformMetricCollector	MockMetricCollector
(Android Kotlin / iOS Swift)	(Unit Test Stubbing)

PulseRunner Orchestration Engine

- LatencyCollector (high-resolution stopwatch)
- Memory / Thermal / Battery / CPU snapshot capture

Trace Exporters & Analysis Layer

- JsonExporter / CsvExporter / MarkdownExporter

3.1 Telemetry Collection Interface

The core contract is defined by the MetricCollector abstract class in `edgepulse_core`:

```
abstract class MetricCollector {  
    Future<void> initialize();  
    Future<MemorySnapshot> captureMemory();  
    Future<ThermalState> captureThermalState();  
    Future<double?> captureBatteryDrainMah();  
    Future<double?> captureCpuUsagePercent();  
    Future<void> dispose();  
}
```

3.2 Platform Implementation Layer

The native plugin (`packages/edgepulse`) interfaces with OS APIs via Flutter `MethodChannel` (`io.github.edgepulse/metrics`):

- **Android (Kotlin):**
 - *Memory*: `Debug.getPss()` retrieves process Proportional Set Size (PSS) in MB.
 - *Thermal*: `PowerManager.getCurrentThermalStatus()` maps system thermal levels (`nominal`, `fair`, `serious`, `critical`).
 - *Battery*: `BatteryManager.BATTERY_PROPERTY_CURRENT_NOW` reads instantaneous current in microamperes (μA), converted to milliamperes (mA).
 - *CPU*: Parses `/proc/stat` snapshot deltas across user, system, and idle ticks.
- **iOS (Swift):**
 - *Memory*: `mach_task_basic_info` retrieves process resident size.
 - *Thermal*: `ProcessInfo.processInfo.thermalState` reads system thermal state.
 - *Battery*: `UIDevice.current.batteryLevel` reads state of charge.
 - *CPU*: `host_statistics64` calculates system CPU tick distribution.

3.3 Execution Orchestration Engine

The `PulseRunner` orchestrates execution tracing: 1. Performs configured warmup iterations without recording metrics to stabilize memory pools and execution graphs. 2. For each measured iteration, records start timestamp, captures pre-inference memory state, executes the target callback, captures post-inference memory and peak RSS, samples thermal and power states, and packages the result into an `InferenceTrace` object.

4. Experimental Setup and Methodology

4.1 Study Design

All empirical measurements reported in Section 5 were performed on a physical Tecno CH7n smartphone (MediaTek Helio G35 SoC, 8-core ARM Cortex-A53 up to 2.3 GHz, 4 GB LPDDR4X RAM, Android 12, API 31). No mock metric collectors or simulated environments were used for the empirical dataset.

4.2 Benchmark Model Workloads

Three runtime engines and model architectures were evaluated:

1. **TFLite MobileNet-V3-Small**: Image classification model (2.5M parameters, 32-bit floating point quantization).
2. **ONNX Runtime ResNet18**: Computer vision backbone (11.7M parameters, dense convolution graph).
3. **GGUF TinyLlama-1.1B**: Quantized autoregressive language model (1.1B parameters, Q4_K_M 4-bit quantization).

4.3 Test Suites & Telemetry Collection

Measurements were executed using dedicated integration test suites under `packages/edgepulse/example/integration`.

- **Baseline Suite:** 30 isolated inference runs per model architecture under idle initial thermal state ($N = 90$ total baseline traces).
- **Sustained Vision Stress Suite:** 15 minutes of continuous MobileNet-V3-Small inference (4,590 iterations).
- **Sustained LLM Stress Suite:** 15 minutes of continuous TinyLlama 1.1B decoding (441 iterations).

5. Results and Discussion

5.1 Baseline Runtime Characterization

Table 1 summarizes baseline performance metrics across the three evaluated runtimes on physical hardware.

Runtime Engine	Target Model	Parameters	Mean Latency	Latency Range	Peak RSS	Peak Battery Current
TensorFlow Lite	MobileNet-V3-Small	2.5 M	508.4 ms	391–851 ms	275.7 MB	402.8 mA
ONNX Runtime	ResNet18	11.7 M	148.2 ms	128–234 ms	409.4 MB	849.0 mA
GGUF (llama.cpp)	TinyLlama-1.1B Q4	1.1 B	1778.6 ms	1461–2028 ms	904.9 MB	206.1 mA

Table 1: Baseline empirical telemetry on Tecno CH7n ($N = 30$ per runtime).

The baseline data reveals structural execution differences between runtimes. ONNX Runtime achieved a 148.2 ms mean latency on ResNet18 (11.7M parameters), whereas TFLite recorded 508.4 ms on MobileNet-V3-Small (2.5M parameters). This difference stems from multi-threading configuration: ONNX Runtime utilized all 8 CPU cores (drawing up to 849 mA peak battery current), while `tflite_flutter` defaulted to single-threaded execution (drawing 402.8 mA).

5.2 Sustained Thermal Stress Analysis

Table 2 presents results from the two 15-minute continuous inference experiments.

Workload	Total Iterations	Baseline Window Latency ($t \leq 120s$)	Late Window Latency ($t \geq 600s$)	Latency Delta	Reported OS Thermal State
MobileNet-V3-Small	4,590	150.1 ms	143.4 ms	-4.5% (stable)	nominal throughout
TinyLlama-1.1B Q4	441	1829.5 ms	2131.0 ms	+16.5%	nominal throughout

Table 2: 15-minute sustained workload telemetry on Tecno CH7n.

For MobileNet-V3-Small, latency remained stable across 4,590 iterations (rolling mean 143–166 ms). The compute load of depthwise convolutions on a light vision model was insufficient to exceed the thermal dissipation capacity of the Helio G35 SoC.

For TinyLlama 1.1B, continuous GEMM matrix multiplications generated sustained thermal load. Comparing the initial baseline window ($t = 31$ – $121s$, mean 1829.5 ms) with the late execution window ($t = 614$ – $735s$, mean 2131.0 ms), inference latency increased monotonically by +16.5% (a 301.5 ms per-inference penalty).

5.3 The Thermal-API Blind Spot

Throughout the 15-minute TinyLlama experiment, `PowerManager.currentThermalStatus` continuously returned `nominal`.

This observation indicates an OS-level telemetry gap. On budget OEM Android distributions (such as Tecno’s HiOS), the high-level `PowerManager` thermal status API does not surface initial kernel DVFS clock frequency scaling. As a result, software applications relying on OS thermal state callbacks fail to detect actual hardware throttling and latency degradation.

Direct, application-level timing and memory profiling—as implemented in EdgePulse—provides the necessary visibility to detect performance regressions caused by thermal throttling.

6. Limitations

1. **Device Diversity:** Hardware measurements were conducted on a single budget device (Tecno CH7n / MediaTek Helio G35). Behavior on flagship processors with active thermal management requires further study.
2. **Thermal Register Access:** Kernel-level DVFS clock frequencies could not be directly sampled without root access. Throttling is inferred from latency degradation under sustained compute.
3. **iOS Platform Validation:** The iOS Swift plugin implementation was validated via unit test stubs rather than physical iOS hardware benchmarks.

7. Conclusion

This paper introduced **EdgePulse**, an open-source runtime observability framework for on-device AI models on consumer edge hardware. By unifying platform-native telemetry (Android Kotlin and iOS Swift) with a Dart execution engine, EdgePulse enables structured monitoring of latency, memory PSS, battery current, and thermal state. Empirical evaluation on physical Android hardware revealed a +16.5% latency degradation during sustained LLM inference that remained undetected by standard OS thermal APIs, highlighting the importance of direct application-level runtime telemetry for edge AI deployments.

References

1. Abadi, M., et al. (2016). TensorFlow: A system for large-scale machine learning. *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, 265–283.
2. Jacob, B., et al. (2018). Quantization and training of neural networks for efficient integer-arithmetic-only inference. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2704–2713.
3. Frantar, E., et al. (2023). GPTQ: Accurate post-training quantization for generative pre-trained transformers. *International Conference on Learning Representations (ICLR)*.
4. Lee, J., et al. (2019). On-device neural network execution for mobile AI applications. *IEEE Circuits and Systems Magazine*, 19(2), 24–39.
5. Geron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O’Reilly Media.
6. David, R., et al. (2021). TensorFlow Lite Micro: Embedded machine learning on TinyML systems. *Proceedings of Machine Learning and Systems (MLSys)*, 3, 800–811.
7. Han, S., Mao, H., & Dally, W. J. (2016). Deep Compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. *International Conference on Learning Representations (ICLR)*.
8. Wang, X., et al. (2024). Thermal-aware dynamic batching and core allocation for mobile LLM inference. *ACM Transactions on Embedded Computing Systems*, 23(4), 1–22.